demo.py

```python

import requests
import streamlit as st
import fitz
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

# ------------------------------
# Configuration
# ------------------------------
OLLAMA_URL = "http://localhost:11434/api/chat"
LLM_MODEL = "llama3.2"
EMBEDDING_MODEL = "BAAI/bge-small-en-v1.5"
CHUNK_SIZE = 900
CHUNK_OVERLAP = 150
TOP_K = 4

st.set_page_config(
    page_title="Medical Document RAG Chatbot",
    page_icon="🩺",
    layout="wide"
)

st.title("🩺 Medical Document RAG Chatbot")
st.caption(
    "Upload a medical PDF and ask questions about its contents."
)

# ------------------------------
# Load embedding model
# ------------------------------
@st.cache_resource
def load_embedding_model():
    return SentenceTransformer(EMBEDDING_MODEL)

# ------------------------------
# Split text into chunks
# ------------------------------
def split_text(text, chunk_size=CHUNK_SIZE,
               overlap=CHUNK_OVERLAP):
    text = " ".join(text.split())
    chunks = []

    start = 0
    while start < len(text):
        end = min(start + chunk_size, len(text))
        chunk = text[start:end].strip()

        if chunk:
            chunks.append(chunk)
```

```
52
53          if end == len(text):
54              break
55
56          start = end - overlap
57
58      return chunks
59
60  # ------------------------------
61  # Read PDF and build FAISS index
62  # ------------------------------
63  def process_pdf(pdf_bytes):
64      pdf = fitz.open(stream=pdf_bytes, filetype="pdf")
65      documents = []
66
67      for page_number, page in enumerate(pdf, start=1):
68          page_text = page.get_text("text")
69
70          for chunk in split_text(page_text):
71              documents.append({
72                  "text": chunk,
73                  "page": page_number
74              })
75
76      pdf.close()
77
78      if not documents:
79          raise ValueError(
80              "No selectable text found. This PDF may be scanned "
81              "and require OCR."
82          )
83
84      embedder = load_embedding_model()
85      texts = [doc["text"] for doc in documents]
86
87      embeddings = embedder.encode(
88          texts,
89          convert_to_numpy=True,
90          normalize_embeddings=True,
91          show_progress_bar=False
92      ).astype("float32")
93
94      # Inner product on normalized vectors = cosine similarity
95      index = faiss.IndexFlatIP(embeddings.shape[1])
96      index.add(embeddings)
97
98      return index, documents
99
100 # ------------------------------
101 # Retrieve relevant passages
102 # ------------------------------
103 def retrieve_context(question, index, documents):
104     embedder = load_embedding_model()
105
```

```
    query_vector = embedder.encode(
        [question],
        convert_to_numpy=True,
        normalize_embeddings=True
    ).astype("float32")

    k = min(TOP_K, len(documents))
    scores, indices = index.search(query_vector, k)

    retrieved = []

    for score, idx in zip(scores[0], indices[0]):
        if idx < 0:
            continue

        retrieved.append({
            **documents[int(idx)],
            "score": float(score)
        })

    return retrieved

# ------------------------------
# Generate answer with Ollama
# ------------------------------
def ask_ollama(question, context):
    prompt = f"""
You are a medical-document research assistant.

Answer the question using ONLY the provided document excerpts.
If the excerpts do not contain the answer, say:
"I could not find this information in the uploaded document."

Do not invent facts, citations, diagnoses, or treatments.
Distinguish information stated by the document from uncertainty.
This tool is for research and education, not medical diagnosis.

DOCUMENT EXCERPTS:
{context}

USER QUESTION:
{question}

Give a clear, concise answer. Cite supporting page numbers
using the provided [Page N] labels.
"""

    response = requests.post(
        OLLAMA_URL,
        json={
            "model": LLM_MODEL,
            "messages": [
                {"role": "user", "content": prompt}
            ],
```

```
            "stream": False
        },
        timeout=180
    )

    response.raise_for_status()
    return response.json()["message"]["content"]

# ------------------------------
# Sidebar: upload document
# ------------------------------
with st.sidebar:
    st.header("Upload document")

    uploaded_file = st.file_uploader(
        "Choose a medical PDF",
        type=["pdf"]
    )

    if st.button("Process PDF", type="primary",
                 use_container_width=True):
        if uploaded_file is None:
            st.warning("Please upload a PDF first.")
        else:
            try:
                with st.spinner(
                    "Extracting text and creating embeddings..."
                ):
                    index, documents = process_pdf(
                        uploaded_file.getvalue()
                    )

                st.session_state.index = index
                st.session_state.documents = documents
                st.session_state.file_name = uploaded_file.name
                st.session_state.messages = []

                st.success(
                    f"Processed {len(documents)} text chunks."
                )

            except Exception as error:
                st.error(f"Could not process PDF: {error}")

    if "file_name" in st.session_state:
        st.info(f"Active PDF: {st.session_state.file_name}")

    st.caption(
        "Large PDFs may take time to process. "
        "Scanned PDFs may require OCR."
    )

# ------------------------------
# Chat interface
```

```python
# ------------------------------
if "messages" not in st.session_state:
    st.session_state.messages = []

if "index" not in st.session_state:
    st.info("Upload and process a PDF to start chatting.")
    st.stop()

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

        if message.get("sources"):
            with st.expander("View retrieved sources"):
                for source in message["sources"]:
                    st.markdown(
                        f"**Page {source['page']}** "
                        f"(similarity: {source['score']:.3f})"
                    )
                    st.write(source["text"])

question = st.chat_input(
    "Ask a question about your uploaded document..."
)

if question:
    st.session_state.messages.append({
        "role": "user",
        "content": question
    })

    with st.chat_message("user"):
        st.markdown(question)

    with st.chat_message("assistant"):
        try:
            with st.spinner("Searching document and generating answer..."):
                retrieved = retrieve_context(
                    question,
                    st.session_state.index,
                    st.session_state.documents
                )

                context = "\n\n".join(
                    f"[Page {item['page']}]\n{item['text']}"
                    for item in retrieved
                )

                answer = ask_ollama(question, context)

            st.markdown(answer)

            with st.expander("View retrieved sources"):
                for source in retrieved:
```

```
                    st.markdown(f"**Page {source['page']}**")
                    st.write(source["text"])

            st.session_state.messages.append({
                "role": "assistant",
                "content": answer,
                "sources": retrieved
            })

        except requests.exceptions.ConnectionError:
            st.error(
                "Cannot connect to Ollama. Open Ollama and "
                "check that the local API is running."
            )
        except requests.exceptions.Timeout:
            st.error(
                "The model took too long to respond. Try a "
                "shorter question or a smaller document."
            )
        except Exception as error:
            st.error(f"Something went wrong: {error}")


```